

12. Fractional Knapsack

```
#include <stdio.h>
```

```
struct Item {
```

```
    int weight; int profit; float ratio;
```

```
};
```

```
void sortItems(struct Item items[], int n) {
```

```
    int i, j;
```

```
    struct Item temp;
```

```
    for(i=0; i<n-1; i++) {
```

```
        for(j=i+1; j<n; j++) {
```

```
            if(items[i].ratio < items[j].ratio) {
```

```
                temp = items[i];
```

```
                items[i] = items[j];
```

```
                items[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
float fractionalKnapsack(struct Item items[], int n, int capacity)
```

```
{
```

```
    float totalProfit = 0.0;
```

```
    sortItems(items, n);
```

```
    for(i=0; i<n; i++) {
```

```
        if(items[i].weight <= capacity) {
```

```
            totalProfit += items[i].profit;
```

```
            capacity -= items[i].weight;
```

```
        }
```

```
    else {
```

```
        totalProfit += items[i].ratio * capacity;
```

```
        break;
```

```
    }
```

```
}
```

```

    return totalProfit;
}

int main() {
    int n, capacity, vi;
    printf("Enter number of items:");
    scanf("%d", &n);
    struct Item items[n];
    printf("Enter weight and profit of each item:\n");
    for (i = 0; i < n; i++) {
        scanf("%d %d", &items[i].weight, &items[i].profit);
        items[i].ratio = (float) items[i].profitweight / items[i].weight;
    }

    printf("Enter knapsack capacity:");
    scanf("%d", &capacity);
    float maxProfit = fractionalKnapsack(items, n, capacity);
    printf("Maximum profit = %.2f\n", maxProfit);
    return 0;
}

```

o/p:

Enter number of items: 4
 Enter weight and profit of each item:

3 42

5 23

2 55

7 12

Enter knapsack capacity: 7

Maximum profit = 106.20

3.16. Remove Duplicate Letters.

```
char* removeDuplicateLetters(char* s)
```

```
{  
    int freq[26] = {0};
```

```
    int visited[26] = {0};
```

```
    int n = strlen(s);
```

```
    for (int i = 0; i < n; i++)
```

```
        freq[s[i] - 'a']++;
```

```
}
```

```
    char* result = (char*) malloc((n+1) * sizeof(char));
```

```
    int top = -1;
```

```
    for (int i = 0; i < n; i++)
```

```
        char ch = s[i];
```

```
        freq[ch - 'a']--;
```

```
        if (visited[ch - 'a'])
```

```
            continue;
```

```
        while (top >= 0 && result[top] > ch && freq[result[top] - 'a'] > 0)
```

```
            visited[result[top] - 'a'] = 0;
```

```
            top--;
```

```
        }
```

```
        result[++top] = ch;
```

```
        visited[ch - 'a'] = 1;
```

```
    }
```

```
    result[top+1] = '\0';
```

```
    return result;
```

```
}
```